

CURSO DE WEB BÁSICO

Materialize CSS – Tutorial Guiado (Passo a passo)

Profº Wagner Wesley Leme Ramalho

E-mail: wagner.ramalho@cps.sp.gov.br

FRAMEWORKS FRONT-END E INTRODUÇÃO AO MATERIALIZE CSS

Objetivos:

- Entender o conceito de framework
- Conhecer diferentes tipos de frameworks
- Compreender os benefícios do uso de frameworks
- Aplicar componentes utilizando Materialize CSS.

O QUE É UM FRAMEWORK?

Um framework é um conjunto de ferramentas, bibliotecas, padrões e componentes prontos que auxiliam no desenvolvimento de software.

Em vez de criar tudo do zero, o desenvolvedor utiliza estruturas já organizadas para acelerar o desenvolvimento.

Construir uma casa:

Sem framework → fabricar cada tijolo

Com framework → utilizar uma planta pronta e materiais padronizados

POR QUE UTILIZAR FRAMEWORKS?

Vantagens

- Maior produtividade
- Redução de código repetitivo
- Padronização visual
- Facilidade de manutenção
- Melhor organização do projeto
- Comunidade ativa
- Documentação pronta

Desvantagens

- Curva de aprendizagem
- Dependência da tecnologia
- Pode adicionar arquivos desnecessários
- Limitações de personalização

VOCÊS CONHECEM ALGUM DESSES FRAMEWORKS?



BACK-END

Responsáveis pela lógica,
regras de negócio e dados.



FRONT-END (CSS)

Responsáveis pela estilização
e aparência das interfaces.



FRONT-END (JAVASCRIPT)

Responsáveis pela interface,
interatividade e experiência do usuário.



TIPOS DE FRAMEWORKS

Frameworks Back-End

Responsáveis pela lógica do sistema.

Exemplos:

- Laravel
- Spring Boot
- ASP.NET Core

Funções:

- Autenticação
- Banco de dados
- Rotas
- APIs
- Segurança

TIPOS DE FRAMEWORKS

Frameworks CSS

Responsáveis pela estilização da aplicação.

Exemplos:

- Materialize CSS
- Bootstrap
- Tailwind CSS

Funções:

- Layout
- Responsividade
- Componentes visuais
- Padronização visual

TIPOS DE FRAMEWORKS

Frameworks Front-End

Responsáveis pela interface e interação do usuário.

Exemplos:

- Angular
- Vue.js
- Next.js

Funções:

- Componentização
- Gerenciamento de estado
- Navegação
- Integração com APIs

DESENVOLVIMENTO TRADICIONAL

VS

DESENVOLVIMENTO COM FRAMEWORK



Mais tempo para desenvolver



Mais código repetitivo e manual



Estrutura definida caso a caso



Maior chance de falhas e inconsistências



Manutenção mais complexa



Desenvolvimento mais rápido



Componentes reutilizáveis e padrões prontos



Estrutura organizada e padronizada



Mais segurança e boas práticas



Manutenção mais simples e eficiente



SEM FRAMEWORK

Você precisa construir tudo do zero.



COM FRAMEWORK

Você usa uma planta pronta para construir melhor e mais rápido.

BIBLIOTECA X FRAMEWORK

Diferente de uma biblioteca (onde você chama o código), em um framework, **o framework chama o seu código**. Isso é conhecido como o princípio de Hollywood ("Não nos ligue, nós ligamos para você"), conceito amplamente discutido por autores de engenharia de software como Martin Fowler.

A Biblioteca é uma Ferramenta: Imagine um martelo ou uma chave de fenda. Você decide quando e onde usá-los para apertar um parafuso.

O Framework é o Projeto do Prédio: Ele já define onde ficam as vigas, os canos e a fiação. Você não escolhe onde a estrutura básica fica, mas você "preenche" os apartamentos com o seu estilo (seu código).

IMPERATIVE VS DECLARATIVE PROGRAMMING

A programação imperativa é como dar instruções passo a passo a um chef sobre como fazer uma pizza. A programação declarativa é como pedir uma pizza sem se preocupar com as etapas necessárias para prepará-la.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <div id="app"></div>
10
11   <script type="text/javascript">
12     const app = document.getElementById("app");
13     // Cria dinamicamente um elemento HTML do tipo <h1>
14     // mas ainda não adiciona ele na página
15     const header = document.createElement("h1");
16     // Cria uma variável contendo o texto que será exibido
17     const text = "Título de exemplo";
18     // Cria um nó de texto com o conteúdo da variável text
19     // Esse texto poderá ser inserido dentro de um elemento HTML
20     const headerContent = document.createTextNode(text);
21     // Adiciona o nó de texto dentro do elemento <h1>
22     header.appendChild(headerContent);
23     // Adiciona o elemento <h1> dentro do elemento com id="app"
24     // fazendo ele aparecer na página
25     app.appendChild(header);
26
27   </script>
28 </body>
29 </html>
```

IMPERATIVA

DECLARATIVA

```
index.html X
index.html > html
2  <html lang="en">
3  <head>
6    <title>Document</title>
7  </head>
8  <body>
9    <div id="app"></div>
10
11    <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
12    <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
13    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
14    <script type="text/babel">
15      /* Isso gera erro na execução - Esse código é JSX - JSX (JavaScript XML)
16      é uma extensão de sintaxe para JavaScript, popularizada pelo React */
17      const app = document.getElementById("app");
18      const root = ReactDOM.createRoot(app);
19      root.render(<h1>Título Exemplo</h1>);
20    </script>
21  </script>
22 </body>
23 </html>
```

O QUE É MATERIAL DESIGN?

Material Design é uma linguagem visual criada pela Google.

Características:

- Design limpo
- Uso de sombras
- Animações suaves
- Componentes padronizados
- Foco em experiência do usuário

O QUE É MATERIALIZE CSS?

Materialize é um framework CSS baseado nos conceitos do Material Design.

Benefícios:

- Fácil utilização
- Componentes prontos
- Responsividade
- Ícones integrados
- Layout moderno

COMO USAR MATERIALIZE?

Basta baixar os arquivos no site do Materialize, ou importar via CDN.



ADICIONANDO O MATERIALIZE AO PROJETO

Podemos usar o materialize local, ou via CDN. Neste exemplo, usaremos CDN. Desta forma, acesse o link: <http://archives.materializecss.com/0.100.2/getting-started.html> e copie os links de incorporação, conforme imagem abaixo:

Baixar

Materialize comes in two different forms. You can select which version you want depending on your preference and expertise. To start using Materialize, all you have to do is download one of the options below.

Materialize

This is the standard version that comes with both the minified and unminified CSS and JavaScript files. This option requires little to no setup. Use this if you are unfamiliar with Sass.

MATERIALIZE 

Sass

Esta versão contém o código com arquivos SCSS. Escolhendo esta versão você tem maior controle sobre os componentes inclusos. Você precisará de um compilador Sass se escolher essa opção.

SOURCE 

CDN

You can find all the versions of the CDN at [cdnjs](https://cdnjs.com).

```
language-markup<!-- Compiled and minified CSS -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/materialize/0.100.2/css/materialize.min.css">

<!-- Compiled and minified JavaScript -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/materialize/0.100.2/js/materialize.min.js"></script>
```

RESULTADO ESPERADO É:

O documento deve ficar da seguinte forma. Note que, o CSS é incorporado no head e o JavaScript, no final do body, antes do fechamento da tag.

```
index.html X
index.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/css/materialize.min.css">
7      <title>Materialize Css</title>
8  </head>
9  <body>
10     <h1>Aprendendo Materialize CSS</h1>
11
12
13
14
15     <script src="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/js/materialize.min.js"></script>
16 </body>
17 </html>
```

ADICIONANDO O GOOGLE ICONS

Disponível da documentação do materialize no link <https://materializecss.com/icons.html>

```
index.html X
index.html > html > head > link
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6
7      <!-- Compiled and minified CSS -->
8      <link
9        rel="stylesheet"
10       href="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/css/materialize.min.css"
11     />
12     <!-- Google Icons -->
13     <link
14       href="https://fonts.googleapis.com/icon?family=Material+Icons"
15       rel="stylesheet"
16     />
17
18   <title>Materialize Css</title>
```

Icons

We have included 932 Material Design Icons courtesy of Google. You can download them directly from the [Material Design specs](#).

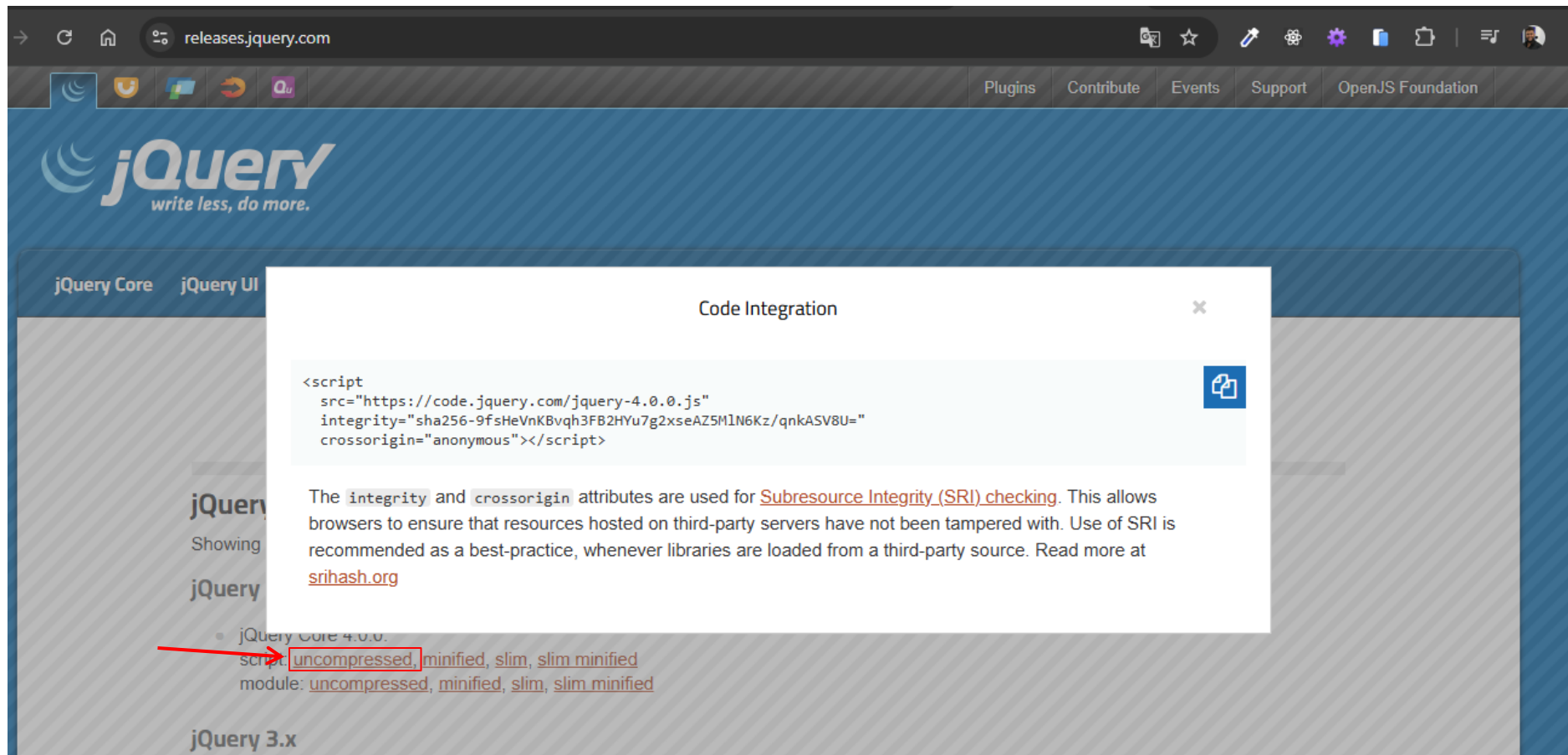
Usage

To be able to use these icons, you must include this line in the `<head>` portion of your HTML code

```
language-markup
<link href="https://fonts.googleapis.com/icon?family=Material+Icons"
```

ADICIONANDO O JQUERY

Disponível no link: <https://releases.jquery.com/>



RESULTADO ESPERADO É:

```
index.html X
index.html > html > body
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6
7      <!-- Compiled and minified CSS -->
8      <link
9        rel="stylesheet"
10       href="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/css/materialize.min.css"
11     />
12     <!-- Google Icons -->
13     <link
14       href="https://fonts.googleapis.com/icon?family=Material+Icons"
15       rel="stylesheet"
16     />
17
18     <title>Materialize Css</title>
19   </head>
20   <body>
21
22     <!-- JQuery CDN -->
23     <script
24       src="https://code.jquery.com/jquery-4.0.0.js"
25       integrity="sha256-9fsHeVnKBvqh3FB2HYu7g2xseAZ5M1N6Kz/qnkASV8U="
26       crossorigin="anonymous"
27     ></script>
28     <!-- Compiled and minified JavaScript -->
29     <script src="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/js/materialize.min.js"></script>
30
31   </body>
32 </html>
```